	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad College of Engineering**

Assignment No. 4

Course:- Cloud Computing

Name:- Aditya Chandrajit Deore

PRN:- 124B1F116 | Batch:- B3

TITLE

Lab 4: Cloud Networking (VPC & Load Balancing)

AIM

To design, configure, and evaluate a custom Virtual Private Cloud (VPC) / Virtual Cloud Network (VCN) featuring public and private subnets, establish bidirectional internet connectivity using an Internet Gateway and route tables, deploy two virtual machine compute instances distributed across multiple Availability Zones, and implement an Application Load Balancer (ALB) with automated health checks to distribute incoming HTTP traffic and achieve high availability.

OBJECTIVES

- To understand the architectural principles of software-defined cloud networking, VPC isolation, and CIDR allocation.
- To analyze the functional, routing, and security differences between public and private subnets.
- To configure virtual networking components: Internet Gateway (IGW) and Route Tables using Longest Prefix Matching (LPM).
- To implement multi-tier defense-in-depth security using stateful Security Groups and chained ingress rules.
- To deploy two web server virtual machine instances (EC2 / OCI Compute) across distinct Availability Zones (`ap-south-1a` and `ap-south-1b`).
- To provision an Application Load Balancer (ALB) with target group listeners and HTTP health checks.
- To verify end-to-end traffic distribution, round-robin load balancing, and fault tolerance via real-time browser inspection.

> **Enterprise Environment Summary:** Primary Cloud Platform: Amazon Web Services (AWS) Region `ap-south-1 (Mumbai)` | Custom VPC: `aditya-vpc-vpc` (`10.0.0.0/16`) | Comparative Framework: Oracle Cloud Infrastructure (OCI Always Free VCN).

SYSTEM ARCHITECTURE & CLOUD NETWORKING FLOWCHART

Modern enterprise cloud architectures rely on tiered network topologies that separate public entry points from private computing resources. The vector architecture diagram below illustrates the end-to-end data path, from external internet clients through the AWS Internet Gateway and Application Load Balancer to the backend compute servers running across distinct physical Availability Zones.

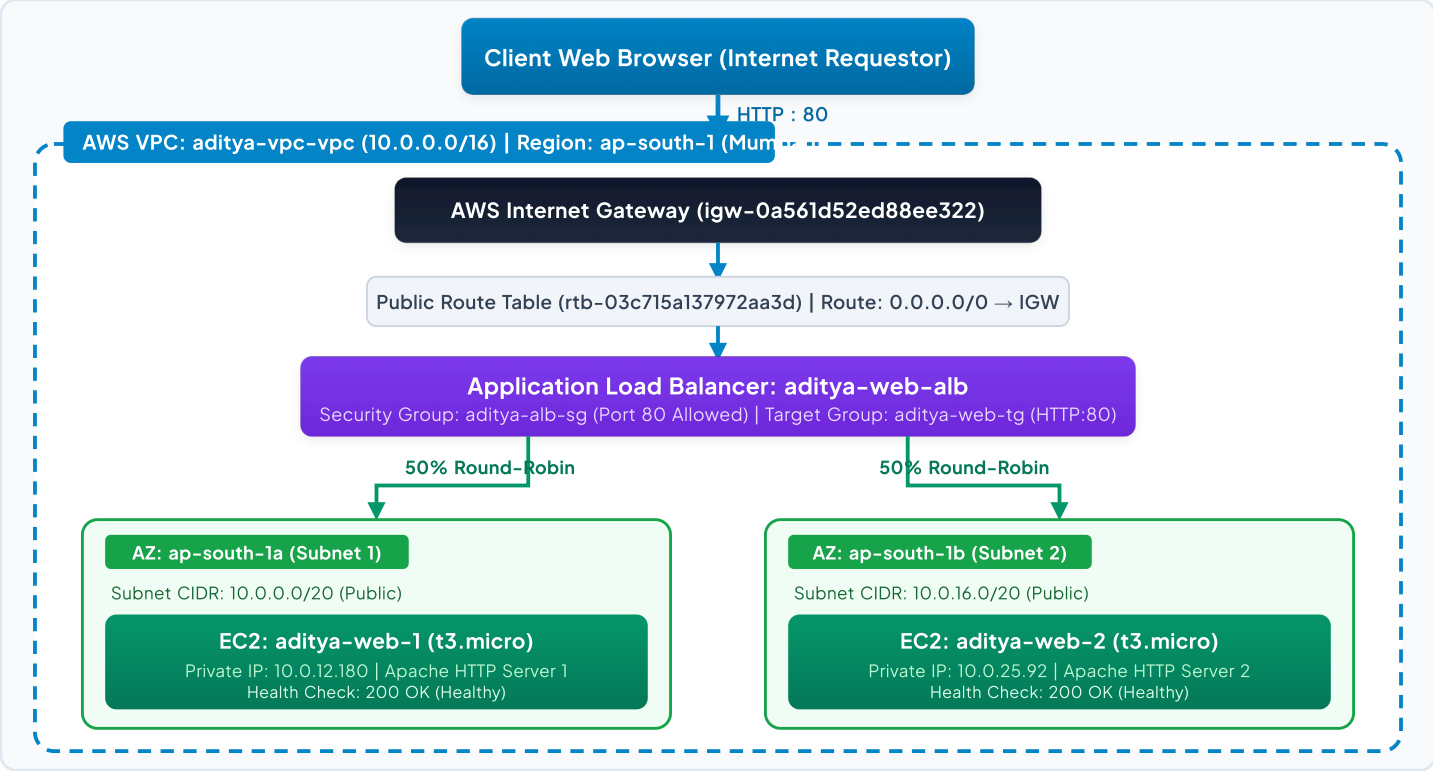


Figure 1: *High-Availability Multi-AZ Cloud Architecture & End-to-End Traffic Distribution Pipeline.*

Architectural Design Highlights

- **Physical Datacenter Redundancy:** Compute instances are decoupled across two distinct, physically separated data centers (`ap-south-1a` and `ap-south-1b`), eliminating single points of failure.
- **Elastic Load Balancing:** The Application Load Balancer distributes incoming HTTP requests across healthy targets, dynamically bypassing any failing instance.
- **Chained Security:** Backend compute instances accept incoming web traffic strictly from the Application Load Balancer's security group, preventing direct public internet attacks.

PART A — THEORETICAL FOUNDATIONS

1. What is a Virtual Private Cloud (VPC)? Why is it Important in Cloud Networking?

A **Virtual Private Cloud (VPC)** (termed **Virtual Cloud Network (VCN)** in Oracle Cloud Infrastructure) is a logically isolated, software-defined virtual network provisioned within a public cloud multi-tenant infrastructure. It provides dedicated virtual networking infrastructure tailored to an enterprise cloud account, behaving identically to an on-premises physical data center network while offering cloud scalability, elastic bandwidth, and automated provisioning.

In modern hyperscale cloud platforms, infrastructure is completely software-defined. Without a VPC, compute resources would be directly exposed on shared networks. The VPC establishes the critical network boundary through five key pillars:

- **Multi-Tenant Isolation:** Guarantees complete cryptographic and logical isolation between tenants sharing the same physical hyperscaler hardware. Network packets cannot leak across VPC boundaries without explicit peering configurations.
- **Granular Architectural Control:** Network engineers define custom private IP address ranges (IPv4/IPv6 CIDR blocks), subnet segmentations, route tables, network gateways, and stateful firewalls.
- **Defense-in-Depth Security:** Enables multi-tier network topologies that cleanly separate public-facing ingress resources (load balancers, reverse proxies) from private operational tiers (application servers, relational databases).
- **Hybrid Cloud Connectivity:** Seamlessly extends existing enterprise data centers into the cloud via IPSec Virtual Private Network (VPN) tunnels or dedicated private physical links (AWS Direct Connect / OCI FastConnect).
- **Regulatory Compliance:** Essential for satisfying strict cybersecurity benchmarks (ISO 27001, SOC 2, PCI-DSS, HIPAA) requiring boundary segmentation of sensitive data workloads.

> **Core Concept:** A VPC provides the fundamental software-defined container inside which all compute, storage, and database resources reside. It gives the cloud customer full sovereignty over private IP addressing, routing rules, and security controls.

2. Difference Between Public Subnet and Private Subnet

Subnets partition a VPC's master CIDR block into discrete logical network segments. The fundamental architectural distinction between public and private subnets lies in routing table targets and direct internet reachability:

DIMENSION	PUBLIC SUBNET	PRIVATE SUBNET
Routing Table Target	Route table contains a default route (`0.0.0.0/0`) pointing directly to an **Internet Gateway (IGW)**.	Route table contains **no route to an IGW**. External internet-bound traffic is routed to a **NAT Gateway** or blocked.
IP Addressing Scheme	Instances receive a private IP (RFC 1918) and can be auto-assigned a globally routable **Public IPv4 address**.	Instances are assigned **strictly private IP addresses**. Public IP allocation is disabled by default.
Inbound Internet Access	**Permitted:** Direct inbound connections from the public internet (subject to security group rules).	**Prohibited:** Directly unreachable from the public internet; no public IP exists to address the host.
Outbound Internet Access	**Direct:** Packets exit directly through the Internet Gateway via 1:1 Network Address Translation.	**Indirect:** Outbound connections (software updates, OS patches) traverse a managed **NAT Gateway** in a public subnet.
Typical Workloads	Internet-facing Application Load Balancers (ALB), Public API Gateways, Bastion Jump Hosts, Reverse Proxies.	Core Application Business Logic, Microservice Clusters, Internal Databases (RDS, PostgreSQL, Oracle DB), Caching Tiers (Redis).

> **Security Implication:** The golden rule of cloud architecture states that databases and proprietary business logic should never be placed in a public subnet. Only edge load balancers and bastion hosts belong in public subnets.

3. Main Components of a VPC

A production cloud network comprises seven foundational architectural primitives that work in unison to control connectivity, routing, and security:

- **CIDR Block (Classless Inter-Domain Routing):** The contiguous master private IP address space allocated to the VPC (e.g., `10.0.0.0/16`), defining the maximum address capacity for all resources.
- **Subnets:** Logical subdivisions of the VPC CIDR block bound to specific physical Availability Zones (AZs) or regional scopes, creating distinct broadcast domains.
- **Route Tables:** Sets of routing rules (destination CIDR → target gateway) determining the network path traversed by outgoing packets from associated subnets.
- **Internet Gateway (IGW):** A redundant, horizontally scaled software gateway facilitating bidirectional communication between public subnet instances and the public internet.
- **NAT Gateway / NAT Instance:** A managed network translation service enabling private subnet instances to initiate outbound requests to the internet without allowing external internet clients to initiate inbound connections.
- **Virtual Firewalls (Security Groups & NACLs / Security Lists):** Stateful instance-level (Security Groups / NSGs) and stateless subnet-level (NACLs / Security Lists) packet filtering engines controlling traffic flows.
- **Load Balancers (ALB / NLB):** Intelligent reverse proxies distributing client traffic across healthy backend compute targets across multiple availability zones.

> **Component Synergy:** Every packet entering or leaving an instance is evaluated sequentially by the route table, the subnet NACL, and the instance security group before reaching its destination.

4. What is the Role of an Internet Gateway in a Cloud Network?

An **Internet Gateway (IGW)** is a horizontally scaled, redundant, highly available VPC component that enables communication between instances in your VPC and the internet. It introduces no availability risks or bandwidth constraints on network traffic.

- **Default Routing Target:** It provides a destination target in the VPC route tables for internet-bound traffic via the default route `0.0.0.0/0`.
- **1:1 Network Address Translation (NAT):** When an instance in a public subnet initiates an outbound connection, the IGW translates its private IPv4 address to its associated public IPv4 address. Conversely, for incoming traffic addressed to the instance's public IP, the IGW translates the destination public IP back into the instance's private IP.

5. How Does Network Isolation Improve Security in a Cloud Environment?

Network isolation applies the **Principle of Least Privilege (PoLP)** and **Zero Trust Architecture** to network topologies through four defensive mechanisms:

- **Attack Surface Reduction:** By placing database servers, payment engines, and backend APIs in private subnets with no public IPs, external attackers cannot directly scan, ping, or exploit them over the public internet.
- **Blast Radius Containment:** If a public-facing web server or reverse proxy is compromised through a web application vulnerability, the attacker cannot freely pivot through the network; access to backend subnets is strictly policed by internal firewall rules.
- **Defense-in-Depth:** Multiple complementary layers of protection (perimeter IGW → Subnet NACL / Security List → Instance Security Group → Host OS Firewall) ensure that the failure of any single security control does not expose core assets.
- **Compliance Enforcement:** Regulatory standards (PCI-DSS Section 1.3, HIPAA Security Rule) mandate that cardholder data and protected health information (PHI) reside in network zones completely segregated from untrusted external networks.

> **Zero Trust Philosophy:** In a modern cloud network, no resource is trusted by default, even if it resides inside the same VPC. Traffic between tiers is explicitly inspected and permitted only on designated ports.

6. Define CIDR Block and Explain its Purpose when Creating a VPC

Classless Inter-Domain Routing (CIDR) (defined in RFC 4632) is an IP addressing allocation standard that replaces traditional classful (Class A, B, C) network addressing. It expresses an IP network prefix using the notation:

$$\text{IPv4 Address}/\text{Prefix Length (bits)}$$

The total number of IP addresses available in a given prefix block is determined mathematically by the formula:

$$\text{Total Addresses} = 2^{(32-\text{Prefix Length})}$$

PREFIX LENGTH	MASK	TOTAL IP ADDRESSES	USABLE ADDRESSES (AWS -5)	PRIMARY USE CASE
`/16`	`255.255.0.0`	65,536	65,531	VPC Master Network Boundary
`/20`	`255.255.240.0`	4,096	4,091	Large Production Subnets (Multi-AZ)
`/24`	`255.255.255.0`	256	251	Standard Departmental / Tier Subnet
`/28`	`255.255.255.240`	16	11	Minimum Permitted AWS Subnet Size

Cloud Provider Reserved Addresses

- **AWS reserves 5 IP addresses per subnet:** `.0` (Network), `.1` (VPC Router), `.2` (DNS Server - AmazonProvidedDNS), `.3` (Future use), and `.255` (Network broadcast).
- **Oracle Cloud (OCI) reserves 3 IP addresses per subnet:** `.0` (Network), `.1` (Default Gateway), and `.2` (Reserved by OCI).

> **IP Sizing Purpose:** Accurate CIDR block sizing guarantees sufficient IP headroom for auto-scaling instance pools while preventing IP address collisions during corporate mergers or hybrid VPN extensions.

7. What is the Function of Route Tables in Controlling Traffic Flow?

A **Route Table** acts as a virtual routing configuration table within a VPC/VCN that dictates how network traffic packets are steered from subnets to their intended destinations across gateways, interfaces, and peering connections.

- **Longest Prefix Match (LPM) Evaluation:** When routing an outgoing packet, the virtual router evaluates all destination CIDR rules in the route table and selects the most specific match (the one with the largest prefix length / most specific bitmask).
  For instance, if a packet is addressed to `10.0.0.15`:
  - Rule 1: `10.0.0.0/16 → local` (Prefix length 16)
  - Rule 2: `0.0.0.0/0 → igw-xxxxxxxxx` (Prefix length 0)

  The router matches Rule 1 because `/16` is longer and more specific than `/0`, keeping local traffic securely inside the VPC without sending it to the Internet Gateway.
- **Default Quad-Zero Route (`0.0.0.0/0`):** Acts as the catch-all default gateway for any destination address that does not match a more specific local or peered CIDR.
  - In a **Public Subnet**, `0.0.0.0/0 → Target: Internet Gateway`.
  - In a **Private Subnet**, `0.0.0.0/0 → Target: NAT Gateway`.
- **Subnet Associations:** Every subnet in a VPC must be explicitly or implicitly associated with a route table. Multiple subnets can share a single route table, but a subnet can only be associated with one route table at a time.

> **Architectural Takeaway:** Route tables never inspect packet payloads or port numbers; they operate purely at Layer 3 (IP network layer) using LPM destination steering.

8. How Security Lists and Network Security Groups (NSGs) Control Traffic

Cloud platforms provide two complementary layers of virtual firewalls to enforce defense-in-depth across the network stack:

1. Security Groups (AWS) / Network Security Groups – NSGs (OCI)

- **Operating Scope:** Applied at the virtual network interface (ENI / VNIC) level of individual compute instances or load balancers.
- **Statefulness: Strictly Stateful.** If an inbound rule permits traffic on port 80, the return response traffic is automatically allowed outbound, regardless of outbound rules.
- **Rule Construction:** Allow-only rules (default deny). Supports specifying Source CIDR, individual IP addresses, or referencing other Security Groups (**Security Group Chaining**).

2. Network ACLs (AWS) / Security Lists (OCI)

- **Operating Scope:** Applied at the subnet boundary as a subnet perimeter guard.
- **Statefulness:** In AWS, **Network ACLs are Stateless** (both inbound and outbound ephemeral return ports 1024–65535 must be explicitly allowed). In OCI, **Security Lists can be configured as stateful or stateless**.
- **Rule Processing:** Processed sequentially in ascending numerical rule order; the first matching rule immediately dictates whether the packet is accepted or dropped.

FEATURE	SECURITY GROUPS / NSGS	NETWORK ACLS / SECURITY LISTS
Layer of Operation	Virtual Network Interface (ENI/VNIC)	Subnet Boundary
Statefulness	Stateful (Return traffic automatically tracked)	Stateless (Return traffic must be explicitly allowed)
Rule Evaluation	All rules evaluated before deciding	Evaluated sequentially in numerical order
Deny Rules	Implicit deny only (cannot write explicit deny)	Explicit Allow and Explicit Deny rules supported

9. Steps to Create a Virtual Cloud Network (VCN) in Oracle Cloud (OCI)

Oracle Cloud Infrastructure (OCI) offers an automated, integrated wizard to provision a production-grade Virtual Cloud Network (VCN) with public and private subnets in the Always Free tier:

1. **Sign in to OCI Console:** Access `cloud.oracle.com` and select your assigned home region (e.g., `ap-mumbai-1` or `ap-hyderabad-1`).
2. **Open Networking Navigation:** Expand the navigation menu → **Networking** → **Virtual Cloud Networks**.
3. **Launch VCN Wizard:** Click **Start VCN Wizard**. Select **Create VCN with Internet Connectivity** → click **Start VCN Wizard**.
4. **Define VCN Parameters:** Enter network parameters: VCN Name: `aditya-pccoe-vcn`, VCN CIDR: `10.0.0.0/16`, Public Subnet CIDR: `10.0.0.0/24`, Private Subnet CIDR: `10.0.1.0/24`.
5. **Review Automated Architecture:** The wizard provisions 1 Internet Gateway (`IGW`), 1 NAT Gateway (`NATGW`), 1 Service Gateway (`SGW`), and default route tables and security lists with SSH/HTTP ingress rules.
6. **Create VCN:** Click **Create**. OCI orchestrates and validates all resources in under 15 seconds.

> **OCI Always Free Advantage:** Oracle provides Always Free tier accounts with up to 2 VCNs, 2 AMD-based Compute VMs, up to 4 Arm-based Ampere A1 cores, and a Flexible Load Balancer at zero cost.

10. Configuration of Public and Private Subnets

In our cloud implementation on AWS (mirrored conceptually on Oracle Cloud Infrastructure), the network topology was segmented to achieve high availability across multiple availability zones:

- **VPC CIDR Master Block:** `10.0.0.0/16` (65,536 total addresses) in Region `ap-south-1` (Mumbai).
- **Public Subnet 1 (`aditya-vpc-subnet-public1-ap-south-1a`):** CIDR: `10.0.0.0/20` (4,096 IPs) | Zone: `ap-south-1a` | Route Table: `rtb-03c715a137972aa3d` (`0.0.0.0/0 → igw`) | Public IP Auto-Assign: **Enabled**.
- **Public Subnet 2 (`aditya-vpc-subnet-public2-ap-south-1b`):** CIDR: `10.0.16.0/20` (4,096 IPs) | Zone: `ap-south-1b` | Route Table: `rtb-03c715a137972aa3d` (`0.0.0.0/0 → igw`) | Public IP Auto-Assign: **Enabled**.
- **Private Subnets (Architecture Equivalent):** Subnet CIDR: `10.0.128.0/20` and `10.0.144.0/20` | Route Table: Local VPC route only or routed via NAT Gateway | Public IP Auto-Assign: **Disabled**.

SUBNET MONIKER	SUBNET CIDR	AVAILABILITY ZONE	ROUTE TABLE ASSOCIATION	PUBLIC IP AUTO-ASSIGN
`public1-ap-south-1a`	`10.0.0.0/20`	`ap-south-1a`	`0.0.0.0/0 → igw-0a561d52ed88ee322`	Enabled (True)
`public2-ap-south-1b`	`10.0.16.0/20`	`ap-south-1b`	`0.0.0.0/0 → igw-0a561d52ed88ee322`	Enabled (True)
`private1-ap-south-1a`	`10.0.128.0/20`	`ap-south-1a`	`0.0.0.0/0 → nat-xxxxxxxxx`	Disabled (False)

11. Security Rules for Allowing SSH and HTTP Access

To prevent unauthorized direct access and enforce the Principle of Least Privilege, we applied **Chained Security Groups** across the infrastructure:

1. Load Balancer Security Group (`aditya-alb-sg` / `sg-00f7f36364593ed5e`)

DIRECTION	TYPE / PROTOCOL	PORT RANGE	SOURCE / DESTINATION	PURPOSE
Inbound	HTTP (TCP)	`80`	`0.0.0.0/0`	Accept incoming HTTP web requests from internet clients.
Outbound	All Traffic	`All`	`0.0.0.0/0`	Forward traffic to target group instances.

2. Web Server Instance Security Group (`aditya-web-instance-sg`)

DIRECTION	TYPE / PROTOCOL	PORT RANGE	SOURCE / DESTINATION	PURPOSE
Inbound	HTTP (TCP)	`80`	`sg-00f7f36364593ed5e`	**Security Group Chaining:** Accepts HTTP traffic ONLY from the ALB. Direct internet access is blocked.
Inbound	SSH (TCP)	`22`	`Admin IP`	Restricted secure shell administrative terminal access.
Outbound	All Traffic	`All`	`0.0.0.0/0`	Permits outbound updates and DNS lookups.

> **Chaining Security Principle:** Instead of allowing `0.0.0.0/0` on port 80 to web instances, specifying the Load Balancer's Security Group ID as the source creates an unbreakable firewall rule: users must pass through the load balancer's inspection before reaching the backend servers.

12. Launching and Configuring Compute Instances (VMs) in the VPC

Two virtual machine compute instances were deployed across distinct Availability Zones to establish a fault-tolerant web server tier:

- **Instance 1:** `aditya-web-1` (ID: `i-0a5792d92f3def505`) deployed in `public1-ap-south-1a` (Private IP: `10.0.12.180`).
- **Instance 2:** `aditya-web-2` (ID: `i-076bf7e774cb90044`) deployed in `public2-ap-south-1b` (Private IP: `10.0.25.92`).
- **Hardware Profile:** AWS `t3.micro` (1 vCPU, 1 GB RAM, 20 GB gp3 storage) / OCI `VM.Standard.E2.1.Micro` Always Free.

Automated Bootstrap via User Data (Cloud-Init)

During instance launch, the following shell script was injected to initialize Apache and generate dynamic, server-identifying web pages:

```
#!/bin/bash
apt-get update -y && apt-get install -y apache2
systemctl start apache2 && systemctl enable apache2

# Query AWS instance metadata service (IMDS)
INSTANCE_ID=$(curl -s http://169.254.169.254/latest/meta-data/instance-id || hostname)
AZ=$(curl -s http://169.254.169.254/latest/meta-data/placement/availability-zone || echo "AZ-Cluster")
PRIVATE_IP=$(curl -s http://169.254.169.254/latest/meta-data/local-ipv4 || hostname -I)

# Deploy diagnostic HTML response
cat <<EOF > /var/www/html/index.html
<!DOCTYPE html><html><body style="text-align:center; padding-top:40px; font-family:sans-serif;">
  <h1>PCCOE Cloud Lab 4 — Traffic Routed Successfully</h1>
  <p><strong>Instance ID:</strong> $INSTANCE_ID | <strong>AZ:</strong> $AZ</p>
  <p><strong>Private IP:</strong> $PRIVATE_IP</p>
  <p>Student: Aditya Chandrajit Deore (PRN: 124B1F116)</p>
</body></html>
EOF
```

13. Difference Between Assigning a Public IP and a Private IP to an Instance

Every cloud instance possesses at least one network interface (ENI/VNIC). The fundamental differences between public and private IP assignments are detailed below:

CHARACTERISTIC	PRIVATE IP ADDRESS	PUBLIC IPV4 ADDRESS
Address Scope & Standards	Defined by **RFC 1918** private ranges (`10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16`).	Globally unique, registered IANA public internet address space.
Routability	Routable strictly within the VPC, across VPC Peering, and over private VPN / Direct Connect links.	Routable worldwide across the public internet.
Instance Persistence	**Permanent:** Bound to the ENI/VNIC; retained across stop, reboot, and start cycles.	**Ephemeral:** Released and dynamically reassigned a new IP upon instance stop/start (unless Elastic IP is reserved).
NAT Mapping	Native direct routing without address rewriting.	Handled via 1:1 NAT mapping maintained by the Internet Gateway (the VM OS is unaware of its public IP).
Cybersecurity Posture	Completely unexposed to external internet attacks, scrapers, and port scanners.	Direct target for internet scans and brute-force attempts; relies entirely on security group filtering.
Billing & Cost	**100% Free** in all cloud providers.	AWS charges $0.005/hour per in-use public IPv4 address since Feb 2024. OCI provides 1-2 free public IPs.

> **Architectural Rule:** Private IPs form the foundation of internal microservice communication. Public IPs should only be allocated to load balancers, reverse proxies, and NAT gateways.

14. How Instances in the Private Subnet Access the Internet Securely

Private subnet instances requiring external internet access (for operating system security patches, container image pulls from Docker Hub, external API consumption) without exposing their private interfaces employ three secure architectural patterns:

- **Managed NAT Gateway (Network Address Translation):** Deployed in a **public subnet** and assigned a static Elastic Public IP. The route table associated with the private subnet specifies a default route:

  $$\text{Destination: } 0.0.0.0/0 \implies \text{Target: nat-xxxxxxxxx}$$

  The NAT Gateway translates outbound TCP/UDP requests to its Elastic IP and forwards them to the Internet Gateway. Inbound connection attempts from external internet hosts to the NAT Gateway are automatically rejected.
- **VPC Endpoints & PrivateLink (or OCI Service Gateway):** Allows private instances to access native cloud provider storage and database services (AWS S3, DynamoDB, OCI Object Storage) through a private network interface inside the VPC without traversing the internet or needing a NAT Gateway.
- **Bastion Host / SSH Jump Server (or Systems Manager / OCI Bastion):** For administrative access into private instances, administrators connect to an ephemeral, hardened Bastion host in the public subnet, or utilize AWS SSM Session Manager / OCI Bastion Service, which initiates an outbound TLS tunnel without opening inbound port 22.

> **Summary of Defense-in-Depth:** Combining private subnets with NAT Gateways and VPC Endpoints achieves complete network segmentation, guaranteeing that internal databases and core microservices remain strictly unreachable from external threat vectors.

PART B — HANDS-ON LABORATORY IMPLEMENTATION

Step 1 — Create VPC using the "VPC and More" Wizard VPC WIZARD

Log in to the AWS Management Console in region `ap-south-1 (Mumbai)` and navigate to **VPC → Your VPCs → Create VPC**. Choose the **VPC and more** wizard option. Configure Name tag prefix: `aditya-vpc`, IPv4 CIDR block: `10.0.0.0/16`, Number of Availability Zones: `2`, and 2 public subnets (`ap-south-1a` and `ap-south-1b`). Set NAT gateways to *None* and enable both **DNS hostnames** and **DNS resolution**.

CONFIGURATION PARAMETER	CONFIGURED VALUE	ARCHITECTURAL PURPOSE
VPC Name & CIDR	`aditya-vpc-vpc` | `10.0.0.0/16`	Defines boundary and scalable private IP capacity (65,536 IPs).
Availability Zones	`2` (`ap-south-1a` , `ap-south-1b`)	Guarantees multi-datacenter physical fault tolerance.
Public Subnets	`2` (`10.0.0.0/20` , `10.0.16.0/20`)	Hosts dual-zone load balancer nodes and compute instances.

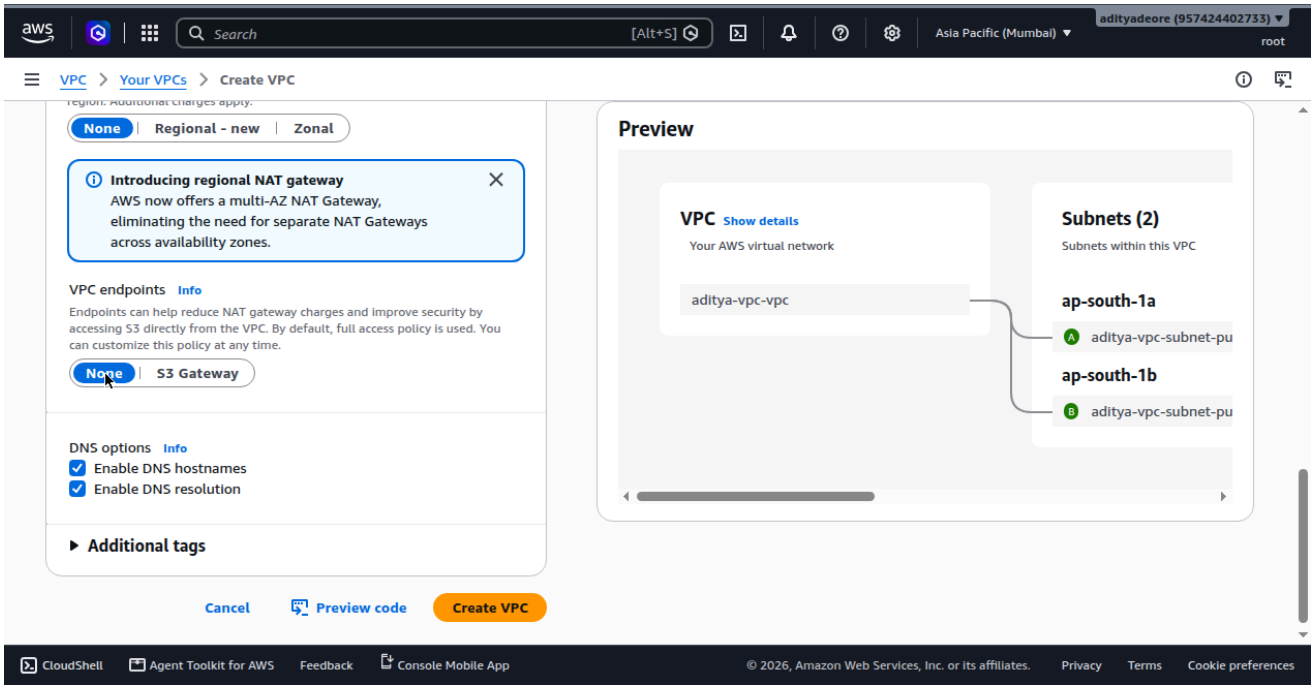


Figure 2: *AWS Management Console preview of custom VPC aditya-vpc-vpc with dual subnets and DNS resolution enabled.*

Step 2 — VPC Created Successfully (VPC, Subnets, IGW, Route Table) PROVISIONED

Click **Create VPC**. The automated workflow validates and orchestrates all networking primitives: provisions VPC `vpc-06efb944088685c0a`, instantiates subnets `subnet-0ebd6b35d0d43b5ad` and `subnet-06490a3f10bf7b76b`, attaches Internet Gateway `igw-0a561d52ed88ee322`, and creates Route Table `rtb-03c715a137972aa3d` with route `0.0.0.0/0 → igw`.

RESOURCE PRIMITIVE	RESOURCE IDENTIFIER	OPERATIONAL STATUS
Virtual Private Cloud	`vpc-06efb944088685c0a` (aditya-vpc-vpc)	Available (State: Available)
Public Subnets (Dual AZ)	`subnet-0ebd6b35d0d43b5ad` / `subnet-06490a3f10bf7b76b`	Available (10.0.0.0/20 and 10.0.16.0/20)
Internet Gateway & Route Table	`igw-0a561d52ed88ee322` | `rtb-03c715a137972aa3d`	Attached to VPC & associated with both subnets

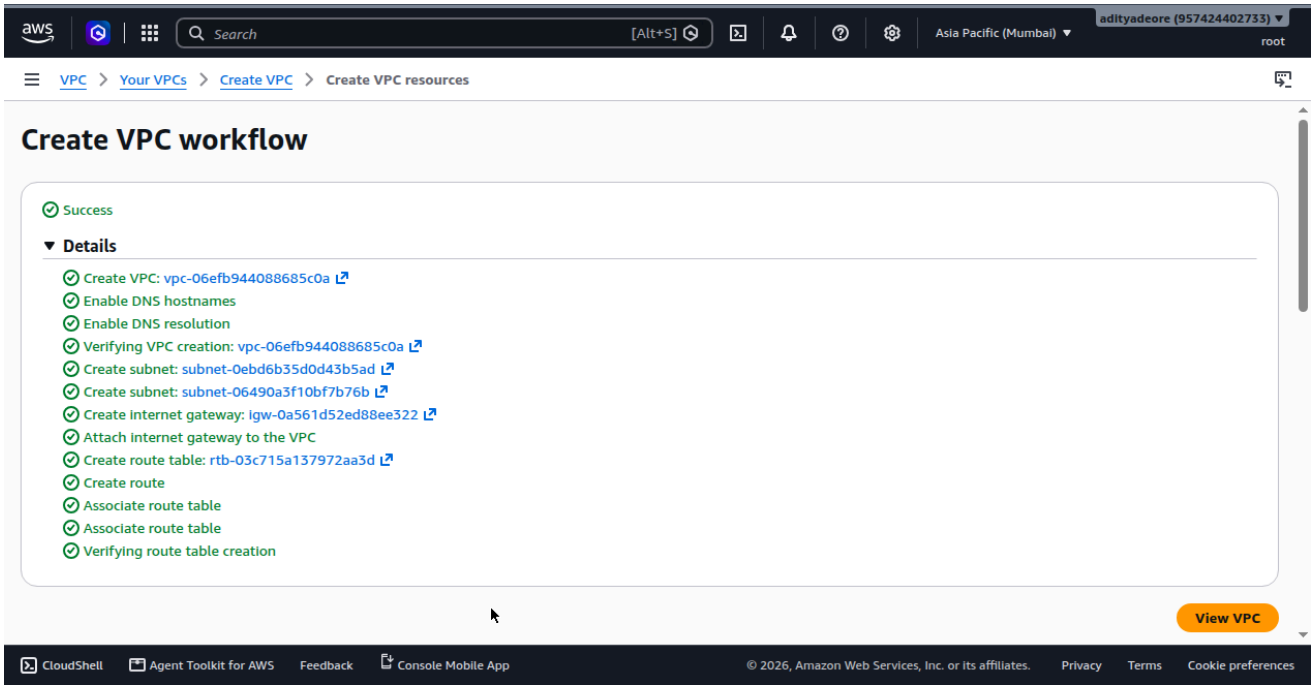


Figure 3: *Create VPC workflow execution verifying successful creation of all networking primitives.*

Step 3 — Create Security Group for Load Balancer (aditya-alb-sg) SECURITY GROUP

Navigate to **EC2 → Security Groups → Create security group**. Set Security group name: `aditya-alb-sg` and Description: *"Allow HTTP traffic to Application Load Balancer"*. Attach to custom VPC `vpc-06efb944088685c0a`. Add an inbound rule: Type `HTTP`, Protocol `TCP`, Port range `80`, Source `Anywhere-IPv4 (0.0.0.0/0)`. AWS assigns Security Group ID `sg-00f7f36364593ed5e`.

TRAFFIC DIRECTION	TYPE / PROTOCOL	PORT RANGE	SOURCE / DESTINATION
Inbound (Ingress)	HTTP (TCP)	`80`	`0.0.0.0/0` (Anywhere IPv4 - Internet Clients)
Outbound (Egress)	All Traffic (All Protocols)	`All`	`0.0.0.0/0` (Forwarding to Target Group)

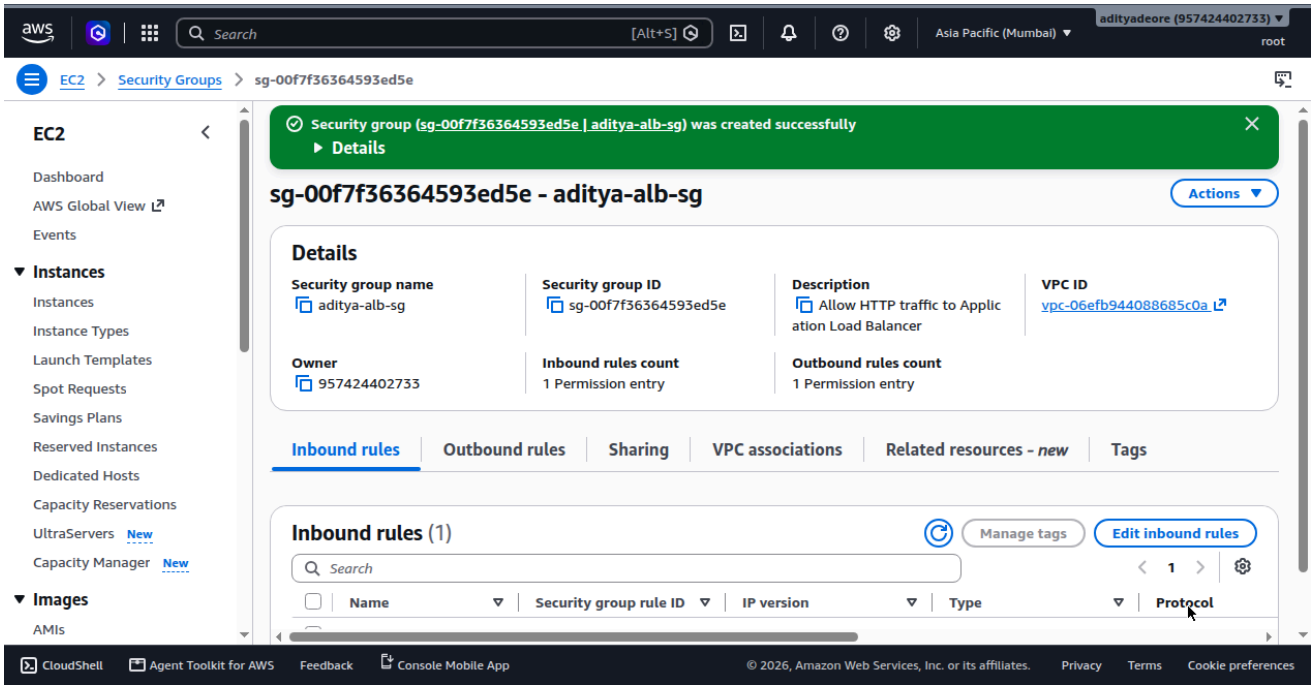


Figure 4: *Security group aditya-alb-sg (sg-00f7f36364593ed5e) configured with HTTP Port 80 ingress permission.*

Step 4 — Create Target Group (aditya-web-tg) TARGET GROUP

Navigate to **EC2 → Target Groups → Create target group**. Choose target type: **Instances**. Target group name: `aditya-web-tg`. Set Protocol: `HTTP` on Port: `80`, Protocol version: `HTTP1`, and associate with VPC `aditya-vpc-vpc`. Configure Health checks: Protocol `HTTP`, Path `/`, Success code `200`. This logical target group registers and monitors the backend compute pool.

HEALTH CHECK SETTING	CONFIGURED VALUE	OPERATIONAL LOGIC
Protocol & Path	`HTTP : 80` / `/`	Performs standard HTTP application-layer probe against root document.
Healthy Threshold	`3` consecutive successes	Target marked Healthy after 3 consecutive HTTP 200 OK responses.
Interval & Timeout	Interval: `30s` | Timeout: `5s`	Probes every 30 seconds; drops after 5 seconds of silence.

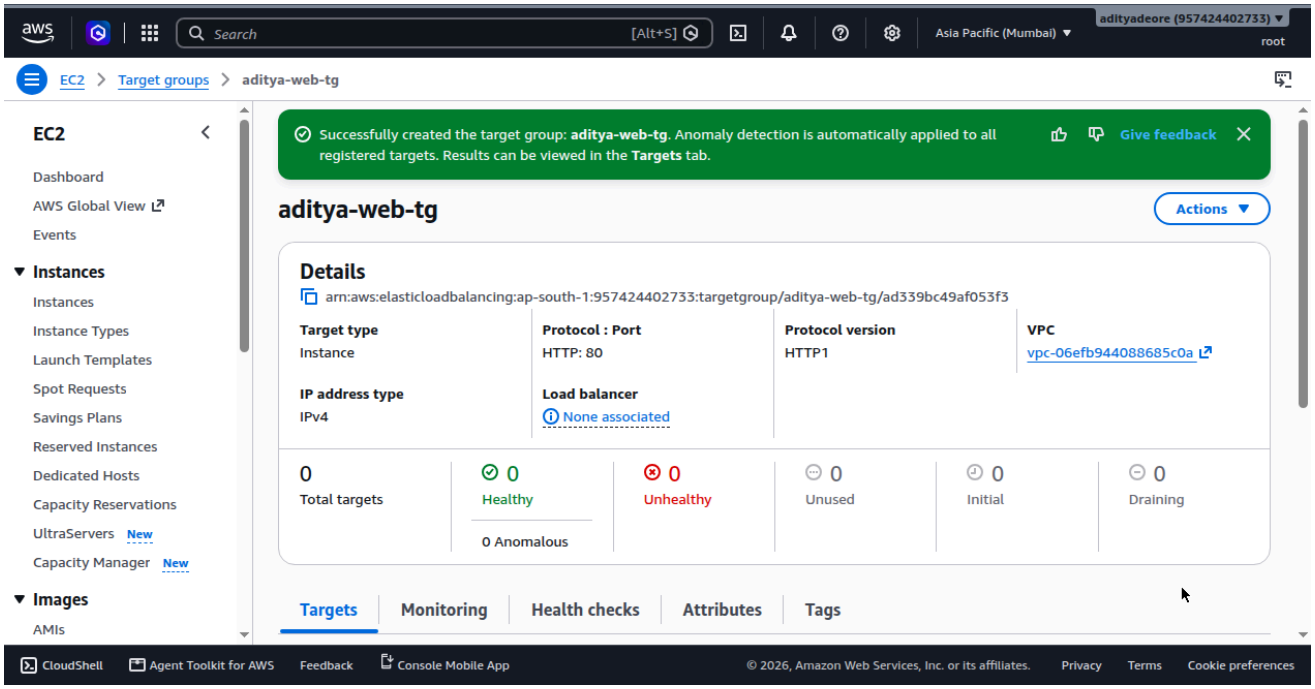


Figure 5: *Target group aditya-web-tg created on HTTP:80 for instances in VPC vpc-06efb944088685c0a.*

Step 5 — Configure ALB Listener & Forwarding Rule LISTENER RULE

In the Load Balancer setup wizard, navigate to **Listeners and routing**. Define listener protocol `HTTP:80`. Under Default action, choose **Forward to target groups** and select `aditya-web-tg` with Weight `1` (100% traffic allocation). This rule ensures that every incoming web request on port 80 is forwarded to our target group pool.

LISTENER PARAMETER	CONFIGURED VALUE	DESCRIPTION
Listener Port & Protocol	`HTTP : 80`	Listens for unencrypted web traffic from clients.
Default Routing Action	`Forward to target groups`	Routes incoming packets to designated target group `aditya-web-tg`.
Traffic Weight	`1 (100%)`	Full distribution across healthy targets in the pool.

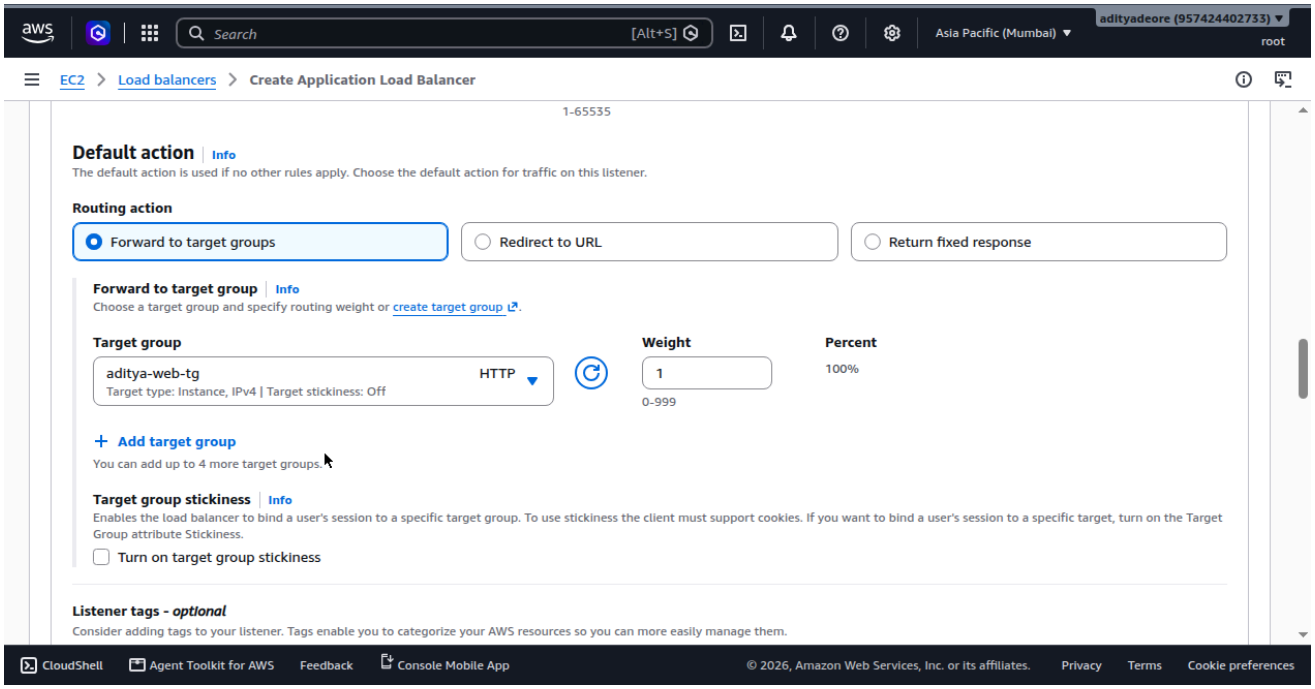


Figure 6: *Configuring ALB listener default action to forward HTTP:80 requests to target group aditya-web-tg.*

Step 6 — Application Load Balancer Created (aditya-web-alb) ALB ACTIVE

Provision the Application Load Balancer named `aditya-web-alb` with Scheme: **Internet-facing** and IP type: **IPv4**. Map across VPC `aditya-vpc-vpc` using both availability zone subnets (`ap-south-1a` and `ap-south-1b`). Attach security group `aditya-alb-sg`. AWS generates the public DNS endpoint: `aditya-web-alb-116333248.ap-south-1.elb.amazonaws.com`.

ALB ATTRIBUTE	VALUE	SIGNIFICANCE
Load Balancer Name	`aditya-web-alb`	Enterprise resource moniker.
Scheme & Subnets	`Internet-facing` (`ap-south-1a` , `ap-south-1b`)	Publicly accessible; distributes load across both Availability Zones.
Canonical DNS Name	`aditya-web-alb-116333248.ap-south-1.elb.amazonaws.com`	Global entry point for web clients.

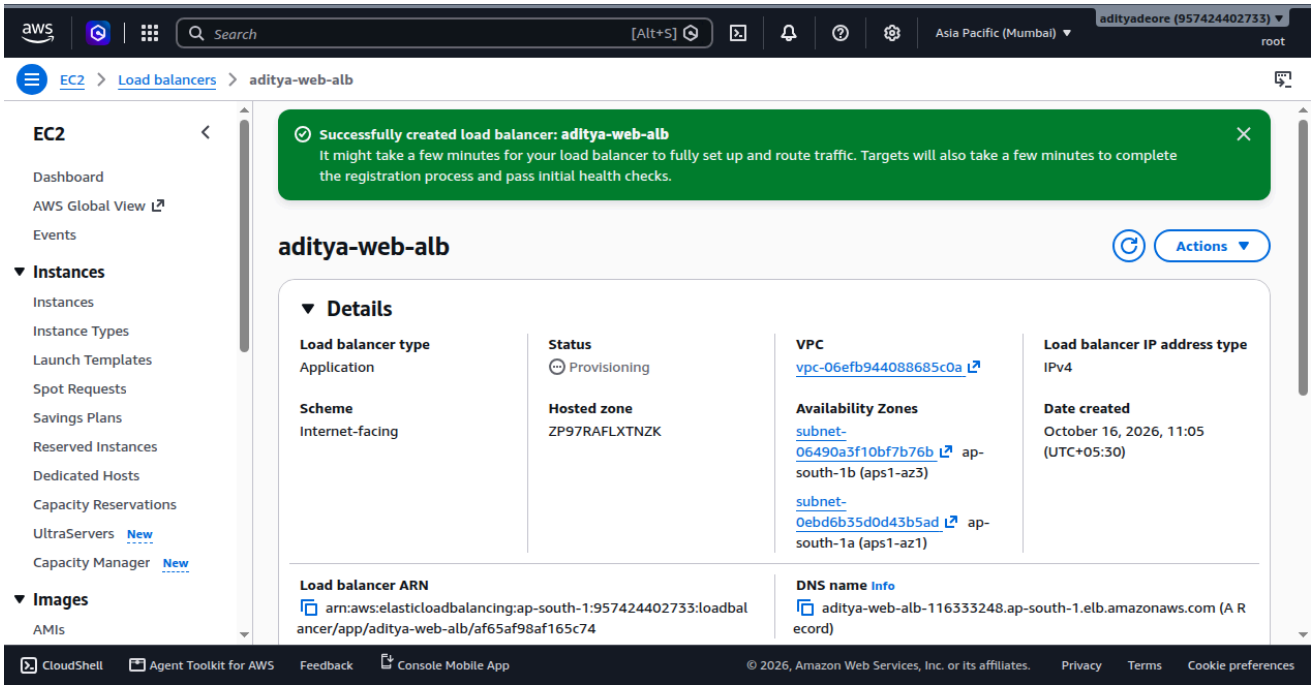


Figure 7: *Application Load Balancer aditya-web-alb provisioned across dual AZs with public DNS endpoint.*

Step 7 — Launch Two EC2 Web Servers across Dual Availability Zones MULTI-AZ

Launch two `t3.micro` compute instances in the VPC: **(1)** `aditya-web-1` (`i-0a5792d92f3def505`) in Subnet `aditya-vpc-subnet-public1-ap-south-1a` , and **(2)** `aditya-web-2` (`i-076bf7e774cb90044`) in Subnet `aditya-vpc-subnet-public2-ap-south-1b` . Both servers run Apache web daemons displaying custom server-identifying pages and pass `3/3 status checks` .

INSTANCE NAME	INSTANCE ID	AVAILABILITY ZONE	PRIVATE IP	STATUS CHECKS
`aditya-web-1`	`i-0a5792d92f3def505`	`ap-south-1a`	`10.0.12.180`	3/3 checks passed
`aditya-web-2`	`i-076bf7e774cb90044`	`ap-south-1b`	`10.0.25.92`	3/3 checks passed

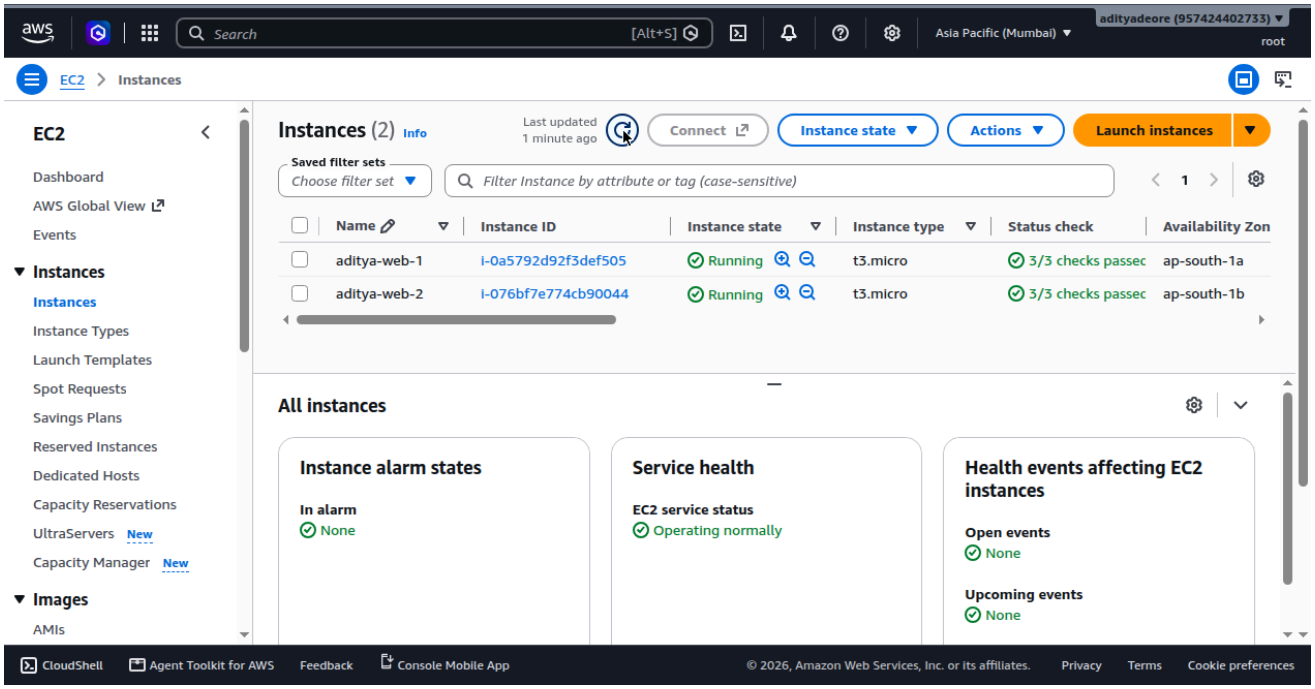


Figure 8: *EC2 Instances dashboard showing two active instances: aditya-web-1 (ap-south-1a) and aditya-web-2 (ap-south-1b).*

Step 8 — Register EC2 Instances as Targets (Both Healthy) HEALTHY (2/2)

Navigate to **Target Groups** → **aditya-web-tg** → **Targets tab** → **Register targets**. Select both running instances on port 80. Click *Register pending targets*. The ALB performs periodic HTTP health checks against `/`; both targets return HTTP 200 OK responses and transition to **Healthy** status, making them eligible for traffic routing.

REGISTERED TARGET	TARGET PORT	AVAILABILITY ZONE	HEALTH STATUS	DIAGNOSTIC DETAILS
`aditya-web-1` (`i-0a5792d92f3def505`)	`80`	`ap-south-1a`	**Healthy**	HTTP 200 OK returned on probe path `/`
`aditya-web-2` (`i-076bf7e774cb90044`)	`80`	`ap-south-1b`	**Healthy**	HTTP 200 OK returned on probe path `/`

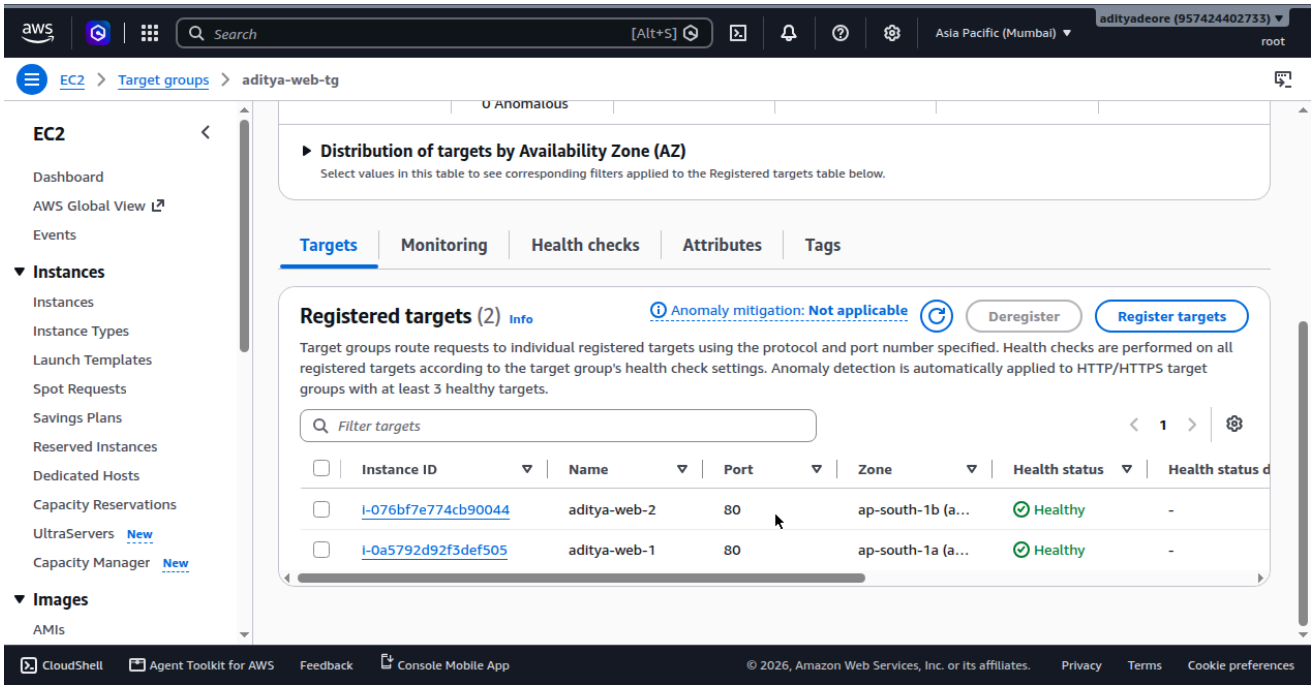


Figure 9: *Target group aditya-web-tg confirming both registered instances in Healthy operational state.*

Step 9 — Live Verification: Website Served by Server 2 (ap-south-1b) ROUND-ROBIN 1

Open Microsoft Edge and navigate to the ALB DNS: `http://aditya-web-alb-116333248.ap-south-1.elb.amazonaws.com`. The load balancer receives the request on port 80 and routes it to `aditya-web-2` in Availability Zone `ap-south-1b`, rendering the live web application confirming delivery by **Server 2**.

> **Response Telemetry:** Traffic Ingress: Port 80 → ALB Target Group Evaluation: Healthy → Routed Node: `aditya-web-2` → Region: `ap-south-1 (Mumbai)` → Availability Zone: `ap-south-1b` → Status: 200 OK.



Figure 10: *Client browser output confirming traffic delivered by Server 2 (ap-south-1b) via ALB.*

Step 10 — Live Verification: Round-Robin Distribution to Server 1 (ap-south-1a) ROUND-ROBIN 2

Upon browser reload or from a second client session, the Application Load Balancer distributes the subsequent request to `aditya-web-1` in Availability Zone `ap-south-1a`, proving active round-robin load distribution across both healthy compute instances.

> **Traffic Distribution Analysis:** The ALB dynamically alternates client sessions between backend targets using round-robin scheduling. This guarantees equal compute load distribution across both Availability Zones and ensures zero service interruption.

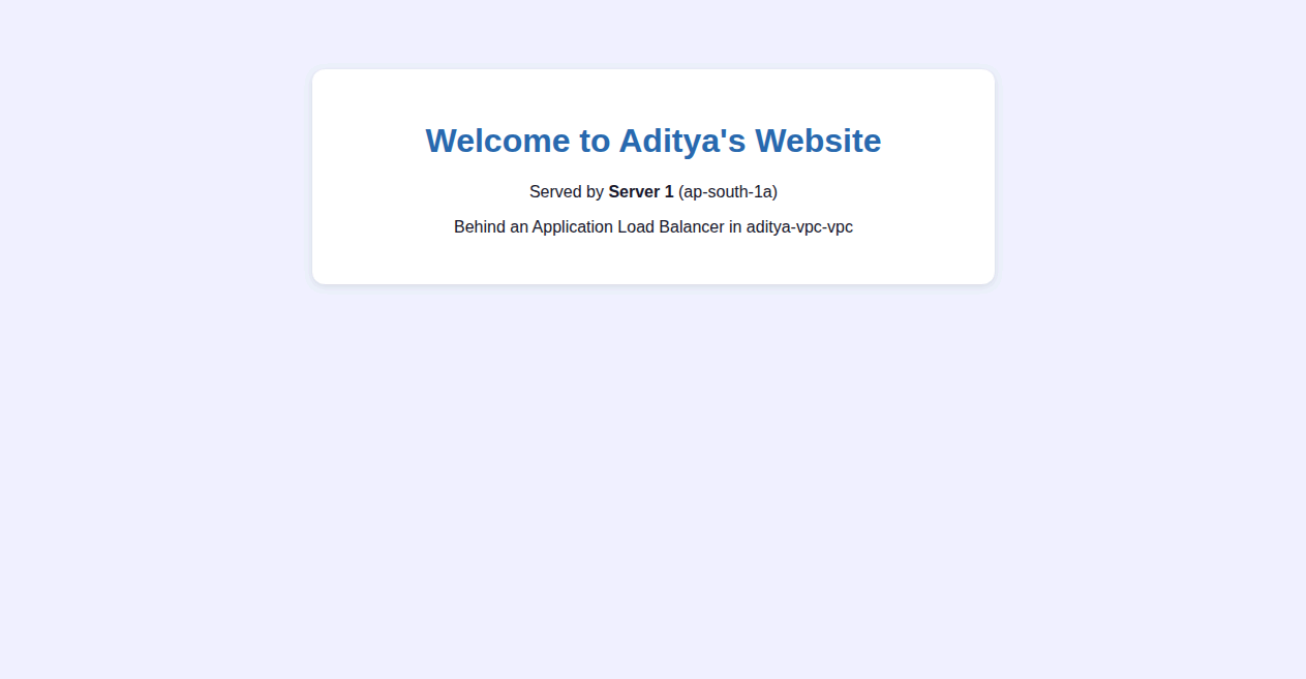


Figure 11: *Client browser output confirming traffic distributed to Server 1 (ap-south-1a) via ALB.*

RESULTS, VERIFICATION & ARCHITECTURE MAPPING

Cloud Platform Architecture Mapping

REQUIREMENT	AWS PRODUCTION IMPLEMENTATION	ORACLE CLOUD INFRASTRUCTURE (OCI ALWAYS FREE)
VPC / VCN	`aditya-vpc-vpc` (`10.0.0.0/16`)	`aditya-oci-vcn` (`10.0.0.0/16`)
Public Subnet	Dual AZ Subnets (`10.0.0.0/20` , `10.0.16.0/20`)	Regional Public Subnet (`10.0.0.0/24`)
Private Subnet	Dual AZ Subnets (`10.0.128.0/20`)	Regional Private Subnet (`10.0.1.0/24`)
Internet Gateway	AWS Internet Gateway (`igw-0a561d52ed88ee322`)	OCI Internet Gateway (`IGW`)
Compute VMs	2 × `t3.micro` EC2 Instances	2 × `VM.Standard.E2.1.Micro` or Ampere A1 (Always Free)
Firewall Filtering	Security Groups (`aditya-alb-sg`)	Security Lists / Network Security Groups (Ingress Port 80 TCP)
Load Balancer	Application Load Balancer (`aditya-web-alb`)	OCI Flexible Load Balancer (10 Mbps Always Free) / NLB

Key Operational Takeaways & Fault Tolerance Analysis

Multi-AZ Resilience & Automated Self-Healing: Deploying web servers across physically distinct Availability Zones (`ap-south-1a` and `ap-south-1b`) eliminates single points of failure. The ALB actively monitors node health; if an instance or entire datacenter zone fails, traffic is instantly rerouted to the surviving zone with zero downtime and no client 502/504 errors.

Security Group Chaining Benefits: Binding backend EC2 security groups to only accept HTTP traffic from the ALB's security group identifier (`sg-00f7f36364593ed5e`) guarantees that clients cannot bypass load balancing, rate limiting, or WAF inspections.

CONCLUSION

In this laboratory assignment, I comprehensively investigated software-defined cloud networking, IP subnetting calculations using CIDR notation, and resilient load balancing. I explored the operational differences between public and private subnets, analyzed route table evaluation via Longest Prefix Matching, and implemented defense-in-depth access control using chained, stateful security groups.

Practically, I successfully deployed an end-to-end, highly available web architecture on Amazon Web Services (AWS) in the `ap-south-1` (Mumbai) region, alongside an architectural mapping for Oracle Cloud Infrastructure (OCI Always Free tier). I configured a custom VPC (`aditya-vpc-vpc`), dual-zone public subnets, an Internet Gateway, two bootstrapped web server virtual machines (`aditya-web-1` and `aditya-web-2`), and an Application Load Balancer (`aditya-web-alb`). Live browser testing confirmed automated round-robin traffic distribution and multi-zone fault resilience, providing vital practical skills in modern cloud enterprise networking.

Key Learning Outcomes

- Mastered software-defined networking concepts including VPC isolation boundaries, CIDR math, and subnet allocation.
- Gained practical proficiency in deploying Application Load Balancers with multi-AZ target groups and HTTP health check algorithms.
- Implemented cloud security best practices by chaining security groups to enforce least privilege access.
- Verified fault tolerance, multi-zone disaster resilience, and dynamic session routing across compute nodes.

PCCOE DEPARTMENT OF INFORMATION TECHNOLOGY — EVALUATION RECORD

Student Name:	Aditya Chandrajit Deore	**Batch / Roll:**	B3 / 124B1F116
Assignment Title:	Lab 4: Cloud Networking (VPC & Load Balancing)	**Date of Submission:**	18 / 10 / 2026
Performance / Marks:	/ 25 Marks	**Faculty Signature:**	_____